Unidade 03 | Capítulo 02

Operações CRUD com JDBC

Prof.: Allberson Dantas





Sumário

- Execução de comandos SQL: INSERT, SELECT, UPDATE, DELETE;
- Uso de PreparedStatement e ResultSet;
- Tratamento de erros e exceções com SQL;

O que é CRUD?

Na aula anterior, aprendemos a conectar uma Aplicação Java a um Banco de Dados. Agora vamos aprender sobre CRUD.

CRUD é um acrônimo usado no desenvolvimento de software para representar as quatro operações fundamentais sobre dados em sistemas persistentes:

- Create → Inserir novos dados (comando INSERT);
- Read → Consultar dados existentes (SELECT);
- Update → Alterar registros existentes (UPDATE);
- Delete → Remover dados (DELETE).

Criando Projeto

Inicialmente é feito a criação de um Projeto Java, como antes. Configuramos o arquivo pom.xml para adicionar o driver para conecta Java ao MySQL. Criamos também o mesmo código para ConexaoFactory:

```java
package com.mycompany.ex50crud;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexaoFactory {
    private static final String URL = "jdbc:mysql://localhost:3306/meubanco";
    private static final String USUARIO = "root";
    private static final String SENHA = "sua_senha";

    public static Connection conectar() throws SQLException {
        return DriverManager.getConnection(url:URL, user:USUARIO, password:SENHA);
    }
}
```

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/20
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.mycompany</groupId>
    <artifactId>Ex50Crud</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>
    <dependencies>
        <dependency>
            <groupId>mysql</groupId>
            <artifactId>mysql-connector-java</artifactId>
            <version>8.0.33</version>
        </dependency>
    </dependencies>
    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.release>21</maven.compiler.release>
        <exec.mainClass>com.mycompany.ex50crud.Ex50Crud</exec.mainClass>
    </properties>
</project>
```

Material de Apoio: Para rodar o código, execute o projeto da pasta Exemplos: Ex50Crud

Create — Inserindo Dados no Banco

Vamos começar pela operação Create, que insere registros no banco. Com JDBC, fazemos isso com o comando SQL INSERT INTO, que é executado por um objeto PreparedStatement (Vamos utilizar o mesmo Banco de Dados da aula anterior):

```java
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;


public class Ex50Crud {

    public static void main(String[] args) throws SQLException {
        Connection conexao = ConexaoFactory.conectar();

        String sql = "INSERT INTO usuarios (id_usuario, nome, email) VALUES (?, ?, ?)";
        PreparedStatement stmt = conexao.prepareStatement(sql);

        stmt.setInt(parameterIndex: 1, x: 3); // valor manual para o id_usuario
        stmt.setString(parameterIndex: 2, x: "Maria");
        stmt.setString(parameterIndex: 3, x: "maria@email.com");

        stmt.executeUpdate();
        stmt.close();
    }
}
```

id_usuario	nome	email
1	Ana Souza	ana@email.com
2	João Lima	joao@email.com
3	Maria	maria@email.com
NULL	NULL	NULL

Material de Apoio: Para rodar o código, execute o projeto da pasta Exemplos: Ex50Crud

Read — Lendo Dados do Banco

Agora vamos ler os dados salvos no banco com o comando SELECT. Para isso, usamos o PreparedStatement e o objeto ResultSet, que armazena os dados retornados:

```
//READ
String sql = "SELECT * FROM usuarios";
PreparedStatement stmt = conexao.prepareStatement(sql);
ResultSet rs = stmt.executeQuery();

while (rs.next()) {
    System.out.println("ID: " + rs.getInt(columnLabel:"id_usuario"));
    System.out.println("Nome: " + rs.getString(columnLabel:"nome"));
    System.out.println("Email: " + rs.getString(columnLabel:"email"));
}
rs.close();
stmt.close();
```

```
ID: 1
Nome: Ana Souza
Email: ana@email.com
ID: 2
Nome: Jo�o Lima
Email: joao@email.com
ID: 3
Nome: Maria
Email: maria@email.com
```

CAPACITA iRede

Material de Apoio: Para rodar o código, execute o projeto da pasta Exemplos: Ex50Crud

Update — Atualizando Dados

Se quisermos alterar dados, usamos o comando UPDATE. É semelhante ao INSERT, mas com a cláusula WHERE para especificar qual registro será alterado:

```
//UPDATE
String sql = "UPDATE usuarios SET email = ? WHERE id_usuario = ?";
PreparedStatement stmt = conexao.prepareStatement(sql);
stmt.setString(parameterIndex: 1, x: "maria.nova@email.com");
stmt.setInt(parameterIndex: 2, x: 3);
stmt.executeUpdate();
stmt.close();
```

id_usuario	nome	email
1	Ana Souza	ana@email.com
2	João Lima	joao@email.com
3	Maria	maria@email.com

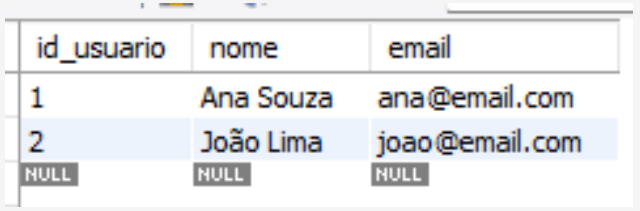
Observe que o email é alterado.

id_usuario	nome	email
1	Ana Souza	ana@email.com
2	João Lima	joao@email.com
3	Maria	maria.nova@email.com
NULL	NULL	NULL

Delete — Removendo Dados

E para excluir um dado do banco, usamos o banco, usamos o comando DELETE. Assim como no UPDATE, a cláusula WHERE é essencial para evitar que todos os registros da tabela sejam apagados:

```
//DELETE
String sql = "DELETE FROM usuarios WHERE id_usuario = ?";
PreparedStatement stmt = conexao.prepareStatement(sql);
stmt.setInt(parameterIndex: 1, x: 3);
stmt.executeUpdate();
stmt.close();
```



id_usuario	nome	email
1	Ana Souza	ana@email.com
2	João Lima	joao@email.com
NULL	NULL	NULL

Tratamento de Erros e Exceções com SQL

Por que tratar erros?

Falhas podem acontecer por diversos motivos:

- Erro de sintaxe na instrução SQL;
- Banco indisponível;
- Tentativa de inserir dados inválidos ou repetidos;
- Problemas de autenticação;
- Conexão encerrada inesperadamente.

Tratando com SQLException:

SQLException é a exceção lançada quando ocorre qualquer erro relacionado ao banco de dados durante o uso da API JDBC.

Tratamento de Erros e Exceções com SQL

```java
//Bloco Try-catch
try {
    Connection conn = ConexaoFactory.conectar();
    String sql = "SELECT * FROM usuarios";
    PreparedStatement stmt = conn.prepareStatement(sql);
    ResultSet rs = stmt.executeQuery();

    while (rs.next()) {
        System.out.println(x: rs.getString(columnLabel:"nome"));
    }

    rs.close();
    stmt.close();
    conn.close();

} catch (SQLException e) {
    System.out.println("Erro ao executar operação no banco: " + e.getMessage());
    System.out.println("Código de erro: " + e.getErrorCode());
    System.out.println("SQLState: " + e.getSQLState());
}
```

```
--- exec:3.1.0:exec (default-cli) @ Ex50Crud ---
Ana Souza
Jo�o Lima
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

CAPACITA iRede

Material de Apoio: Para rodar o código, execute o projeto da pasta Exemplos: Ex50Crud



Lista de Exercícios e Tarefas

Responda o Quiz do Capítulo.

Desenvolva as questões da TAREFA 17 – Operações CRUD com JDBC

CAPACITA iRede

INOVAÇÃO E TECNOLOGIA

RESIDÊNCIA EM TIC 20



EXECUTOR

iRede
INOVAÇÃO E TECNOLOGIA

INICIATIVA

MCTI FUTURO

COORDENADORA PPI

Softex

MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO

*Este projeto foi apoiado pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248, de 23 de outubro de 1991, no âmbito do PPI-SOFTEX, coordenado pela Softex e publicado Residência em TIC 20, DOU 01245.002631/2023-58.